

DATA ENGINEERING & ANALYTICS

Workforce Intelligence.

From Azure SQL to Lakehouse.

A Microsoft Fabric case study in Bronze-to-Silver pipeline design, PySpark transformation logic, quarantine-driven data quality, and the operational discipline of accounting for every record that enters the lakehouse.

CASE STUDY 04 · CONTENTS

Six sections. Eight lab steps. One clean record per employee.

A technology consulting firm runs its workforce reporting on manual SQL exports and Excel spreadsheets. Forty-eight hour turnarounds. Five duplicate employee_ids. Twenty records to quarantine. The lab moves the data from a well-maintained Azure SQL source through a Bronze-Silver lakehouse, producing one clean record per employee that HR can trust.

01

The Brief

Executive situation, six rules for the lab, and the architecture that replaces the export-and-Excel workflow.

PAGE 03

02

The Context

Three regional hubs, one Azure SQL source, no reporting layer. The current data landscape and how it failed.

PAGE 04

03

The Diagnosis

Four classes of data quality issue. Ten null hire dates, eight null salaries, two invalid salaries, five duplicate employee_ids, name prefixes mixed in.

PAGE 05

04

The Target

Bronze-Silver target architecture, target dataset requirements, and source inventory.

PAGE 06

05

The Lab

Eight steps from Bronze inspection through quarantine, deduplication, enrichment, join, metadata, row-count reconciliation, and a department summary.

PAGES 07 – 09

06

The Bonus & Appendix

Two optional challenges: salary rank and name prefixes. Full dataset reference for both employees and departments. Known data quality issues catalogued.

PAGES 10 – 12

EXECUTIVE SITUATION

From manual SQL exports to a clean Silver layer.

Innovate Solutions is a technology consulting firm operating across three regional hubs: London, New York, and Singapore. Following rapid workforce growth across nine business units, the data infrastructure has not kept pace. Employee records sit in an Azure SQL Database with no reporting layer on top. Every HR report requires a manual SQL export, local cleanup in Excel, and a 48-hour turnaround. By the time the data is ready, it is already stale.

This lab deploys the Workforce Intelligence Engine: a data pipeline built on Microsoft Fabric that moves employee data from Azure SQL through a Bronze-Silver lakehouse architecture, producing clean, trusted data for HR reporting. Bronze captures the raw extract. Silver applies cleaning, deduplication, enrichment, and join logic before the data is used for reporting.

SIX RULES

P

Automated Pipeline

Data moves from Azure SQL to the lakehouse on a schedule. No manual exports or emailed spreadsheets.

B

Bronze Stays Raw

Bronze holds the data exactly as it arrived. The notebook reads from Bronze and never writes back to it.

Q

Quarantine, Never Drop

Bad records go to `silver_quarantine` with a reason. No record is silently removed.

D

One Row Per Employee

Where an `employee_id` appears more than once, keep only the record with the latest `updated_at`.

M

Stamp Every Record

Every Silver record carries `transformed_at`, `employee_source`, and `dept_source` columns.

R

Count Everything

Print Bronze, quarantine, duplicates removed, and Silver counts. Bronze equals the other three added together.

SECTION II · ENTERPRISE CONTEXT

A well-maintained source. An absent engineering layer.

Innovate Solutions operates across AMER (New York), APAC (Singapore, Sydney), and EMEA (London). Workforce data from all three hubs feeds into a single centralised Azure SQL Database. The source system is well-maintained: records are structured, typed, and timestamped. The failure is the complete absence of an engineering layer between that source and the analysts who need its data.

WHAT THE SOURCE IS DOING RIGHT

Records have proper types. Foreign keys resolve. Every row carries an `updated_at` timestamp. The database is doing its job well.

The problem is not the source. The problem is the workflow that has grown around the source: a habit of pulling reports straight out of the day-to-day database, with humans bridging the gap one export at a time.

WHAT THE WORKFLOW IS DOING WRONG

HR teams resort to ad-hoc SQL exports, manual cleaning in Excel, and emailed spreadsheets. The process introduces errors, creates version conflicts, and exposes employee PII with no governance or access controls.

“Keeping employee salary data in email attachments is a risk for compliance, audit, and employee privacy.”

EXHIBIT 01 · CURRENT DATA LANDSCAPE

DOMAIN	SYSTEM OF RECORD & ACCESS	REFRESH LATENCY
EMPLOYEE RECORDS	Azure SQL Database (<code>dbo.employees</code>) accessed via manual SQL export to Excel.	Weekly at best. Always behind.
DEPARTMENT DATA	Lakehouse CSV (<code>departments.csv</code>) uploaded manually, with no clear owner.	Ad-hoc. Hard to tell if it is current.
HR ANALYTICS	Analyst desktops (Excel) doing VLOOKUP across exports.	Static. Each report is its own snapshot.

THE STRATEGIC FRAME

Three regional hubs. One source system that is doing its job correctly. One workflow after it that is not. The fix is to build the missing layer: a Bronze layer for the raw data, a Silver layer for the cleaned data, and a notebook that turns one into the other on a schedule.

SECTION III · CURRENT STATE DIAGNOSIS

Four classes of defect waiting in Bronze.

A review of the current data workflow reveals the issues below in the source data. You will need to account for each of them in your Silver notebook.

EXHIBIT 02 · KNOWN ISSUES IN THE SOURCE DATA

ISSUE	DETAIL	WHERE IT LANDS
NULL FIELDS	10 records have a null <code>hire_date</code> . 8 records have a null <code>salary</code> . Neither can be defaulted.	Quarantine · <code>NULL_FIELD</code>
INVALID SALARIES	1 record has <code>salary = 9,999,999</code> . 1 record has <code>salary = -5,000</code> . Bounds are out of plausible range.	Quarantine · <code>INVALID_SALARY</code>
DUPLICATE IDS	5 <code>employee_id</code> values appear more than once. Older versions were kept after salary updates in Azure SQL. Each occurrence carries a different <code>updated_at</code> .	Deduplicate on latest <code>updated_at</code>
NAME PREFIXES	Approximately 10 percent of employee names carry a leading prefix: Mr., Mrs., Ms., or Dr. The prefix carries information, so it is worth keeping in its own column.	Bonus 02 · <code>name_prefix</code> column

DIAGNOSTIC FINDING

The Bronze layer is never modified. It holds exactly what the Fabric Pipeline received from Azure SQL. All cleaning, quarantining, and enrichment happens exclusively in your Silver notebook. Keeping Bronze untouched means you can rerun the notebook at any time and get the same Silver output.

SECTION IV · TARGET DIRECTION

Pipeline plus notebook. Bronze stores. Silver cleans.

The target architecture replaces the manual export-and-Excel workflow with a two-stage Fabric Data Pipeline. A Copy Activity copies employee records from Azure SQL into the Bronze layer. A Notebook Activity then reads from Bronze, applies cleaning and enrichment logic, and writes the trusted output to Silver. Bronze stays unchanged, and all the cleaning happens in Silver.

EXHIBIT 03 · TARGET DATASET REQUIREMENTS

TABLE	LAYER	WHAT EACH ROW HOLDS	WRITE RULE
bronze_employees	BRONZE	Raw copy of the source table, exactly as the Copy Activity landed it. Never modified after writing.	READ ONLY
silver_quarantine	SILVER	One row per rejected record. Audit trail of every record removed from the clean dataset, with the reason why.	APPEND
silver_employees	SILVER	One row per employee (deduplicated). Cleaned, enriched, and joined dataset. Foundation for all HR reporting.	OVERWRITE

EXHIBIT 04 · SOURCE SYSTEM INVENTORY

SOURCE	OBJECT	KEY COLUMNS & INGESTION METHOD	MODE
Azure SQL Database	dbo.employees	employee_id, employee_name, job_title, department_id, hire_date, salary, updated_at. Fabric Pipeline Copy Activity, full table copy.	FULL LOAD
Fabric Lakehouse Files	departments.csv	department_id, department_name, location, region. Manual upload, read directly by Silver notebook as a static reference.	REFERENCE

THE TWO-STAGE PIPELINE



SECTION V · LABORATORY INSTRUCTIONS

Bronze to Silver. Eight steps.

You are the Data Engineer on the Platform Team at Innovate Solutions. Your mission is to move raw employee data from Bronze to Silver, producing a clean, trusted dataset that HR can use for workforce reporting. Bronze is never modified. All cleaning and quarantining happens in Silver. Always use `display()`. Never use `collect()` or `toPandas()` on large DataFrames.

SETUP & SOURCES

ATTRIBUTE	DETAIL	NOTES
Role	Data Engineer, Platform Team	Innovate Solutions
Sources	bronze_employees · departments.csv	Delta · Files
Imports	from pyspark.sql import functions as F	PySpark

Step 01

Inspect

Inspect Your Sources

Load both source datasets and familiarise yourself with what the pipeline has landed. Understand the shape, volume, and data types before writing any transformation logic.

Expected: both sources loaded and displayed.

Step 02

Quarantine

Handle Invalid Records

Some records cannot be trusted for HR reporting. Move them out of your working dataset and write them to a Silver Delta table called `silver_quarantine`, with a `quarantine_reason` column. Use two rules: `NULL_FIELD` when `hire_date` or `salary` is null, and `INVALID_SALARY` when `salary` is below 0 or equal to 9,999,999. No record should be silently dropped.

Expected: `silver_quarantine` written, with both `NULL_FIELD` and `INVALID_SALARY` visible.

LAB INSTRUCTIONS · CONTINUED

Deduplicate. Then enrich.

The remaining records after quarantine still need to become one row per employee, and they need the extra columns HR will report on. Steps 3 and 4 do both. Step 3 collapses duplicates to the latest `updated_at`. Step 4 adds the columns the source did not provide.

Step 03 Deduplicate

Resolve Duplicates

The same employee may appear more than once in Bronze. HR reports must count each employee exactly once. Where duplicates exist, retain only the most current version of the record. Five duplicate IDs in the source: 1014, 1066, 1139, 1154, 1172.

Expected: each `employee_id` appears exactly once.

Step 04 Enrich

Enrich the Dataset

HR needs additional attributes that do not exist in the source system. Four enrichments, all derived from existing columns:

Hire year. Calendar year extracted from `hire_date`.

Tenure. Complete years since hire. Calculated from `hire_date` against current date.

Tenure group. Write this logic as a reusable Python function. New for under 2 years, Established for 2 to 4 years, Veteran for 5 years and above.

Salary band. Senior for 120,000 and above, Mid for 80,000 to 119,999, Junior below 80,000.

Expected: new columns added. No nulls in any enriched column.

LAB INSTRUCTIONS · CONTINUED

Join, stamp, count, summarise.

The final four steps bring department context into the dataset, stamp the transformation with metadata, write the Silver table, reconcile row counts, and summarise the result for HR. After Step 7, every record that entered Bronze is accounted for: in Silver, in quarantine, or removed as a duplicate.

Step 05 Join

Add Department Context

Employee records in Bronze contain a `department_id` but no department name, location, or region. HR reports require all four attributes. Combine the employee data with the department reference file so every employee record carries full department context.

Expected: department columns added. Every employee has a department name.

Step 06 Stamp & Write

Mark the Transformation & Write Silver

Before writing to Silver, each record must carry metadata that identifies when it was transformed and where each piece of data came from. Add `transformed_at`, `employee_source`, and `dept_source`. Then write the final dataset to `silver_employees`.

Expected: silver_employees written and displayed.

Step 07 Reconcile

Reconcile Row Counts

Print four numbers using f-strings: records in Bronze, records quarantined, duplicates removed, and records in Silver.

Expected: 332 in Bronze = 20 quarantined + 5 duplicates removed + 307 in Silver.

Step 08 Summarise

Summarise by Department

HR wants a quick overview. Show the headcount and average salary for each department.

Expected: 9 rows, one per department with employees. Headcounts add up to 307.

SECTION VI · BONUS CHALLENGES

Two bonus exercises.

The following challenges are optional and read from `silver_employees`. They only display results, so Silver is not changed. Bonus 01 introduces window functions. Bonus 02 practises string functions. Each challenge stands alone.

Bonus 01 Rank

Salary Rank Within Department

HR wants to know where each employee ranks within their department by salary. Produce a ranking that resets for each department. Handle tied salaries fairly: same salary should produce the same rank.

Expected: rank resets to 1 at the start of each new department.

Bonus 02 Text

Split Out Name Prefixes

Some employee names start with Mr., Mrs., Ms., or Dr. Move the prefix into a new `name_prefix` column and keep the rest of the name in `employee_name`. Names without a prefix get an empty `name_prefix`.

Expected: 35 rows with a `name_prefix`. No prefixes left in `employee_name`.

WHAT IS A WINDOW FUNCTION

A window function looks at a group of rows, such as one department, and returns a value for every row in that group. Unlike a group by, no rows are lost. Ranking employees within their department is the most common example.

APPENDIX A · DATASET REFERENCE

employees.

Azure SQL · dbo.employees

The full source schema for the employee dataset, with known data quality issues annotated against each column. Review this section carefully before starting the lab. The issues catalogued here are the ones your Silver notebook must handle.

COLUMN	TYPE	DESCRIPTION	KNOWN ISSUES
employee_id	Integer	Unique employee identifier.	5 duplicates
employee_name	String	Full name of the employee.	~10% prefixes
job_title	String	Employee's current job title.	Clean
department_id	Integer	Foreign key to the departments table.	Clean
hire_date	Date	Date the employee joined Innovate Solutions.	10 nulls
salary	Integer	Annual base salary in USD.	8 nulls + 2 invalid
updated_at	Timestamp	When the record was last modified in Azure SQL.	5 nulls

KNOWN DATA QUALITY ISSUES SUMMARY

10 records: null hire_date → quarantine_reason = NULL_FIELD . 8 records: null salary → NULL_FIELD . 1 record: salary = 9,999,999 → INVALID_SALARY . 1 record: salary = -5,000 → INVALID_SALARY . 5 duplicate employee_id : 1014, 1066, 1139, 1154, 1172 → deduplicate using updated_at . 5 records: null updated_at → no action needed, none of them are duplicates. Approximately 36 records: name prefix present → Bonus 02, extract to name_prefix column.

APPENDIX B · DATASET REFERENCE

departments.

Lakehouse CSV · departments.csv

The reference dataset that gives every employee record its department name, location, and regional hub. Ten departments, four cities, three regions. The data is clean and stable. The Silver join is straightforward; what matters is using a join type that does not silently lose employees if a department code is unrecognised.

COLUMN	TYPE	DESCRIPTION	NOTES
department_id	Integer	Unique department identifier (101 to 110).	JOIN KEY
department_name	String	Name of the business unit.	Mapped
location	String	City where the department is based.	4 cities
region	String	Regional hub: AMER, APAC, or EMEA. Used for regional grouping in Silver reporting.	3 regions

REFERENCE DISTRIBUTION

10

Departments
department_id from 101 through 110. Nine are used by employees. 110 (Executive Office) has none.

4

Cities
London, Singapore, New York, Sydney. The three hubs plus Sydney as a satellite APAC office.

3

Regions
AMER (New York), APAC (Singapore, Sydney), EMEA (London). Used for regional grouping in HR reports.

~36

Prefix records
Approximate count of employee names carrying a Mr., Mrs., Ms., or Dr. prefix that Bonus 02 extracts.

From a manual export workflow to a Silver layer HR can trust.

Build the pipeline that gives HR a workforce dataset they can trust. A Copy Activity from Azure SQL, a read-only Bronze layer, a PySpark notebook that quarantines defects, removes duplicates, enriches with hire year, tenure, tenure group, and salary band, joins department context, stamps metadata, reconciles row counts, and summarises by department. One Bronze table in. Two Silver tables out. Every record accounted for.

WHAT YOU WILL DELIVER

A Fabric Pipeline (Copy Activity + Notebook Activity) and a PySpark notebook that moves data from Bronze to Silver, writes silver_employees and silver_quarantine, and prints reconciliation counts at the end of every run.

HOW YOU WILL BE EVALUATED

Six rules covering automation, raw Bronze, quarantine, deduplication, metadata, and row counts.

WHAT THIS PROVES

You can take a source database and a manual workflow, design the missing engineering layer, and deliver a Silver table that HR, finance, and audit can use without asking how it got there.

WHAT COMES NEXT

Build a Power BI report on top of silver_employees. Then schedule the pipeline to run daily so HR always sees fresh data.

INSTRUCTOR CONTACT

Surya Kumar · suryakumar01985@gmail.com